

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении проектной работы
По дисциплине: «Цифровая аналитика»
На тему: «Палитра контрастности»

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
к.т.н.

_____ Мулюха В.А.

«_____» _____ 2026г.

Содержание

1	Техническое задание	3
	История разработки	9

1 Техническое задание

Проект представляет собой веб-приложение «Палитра контрастности», предназначенное для анализа цветов, генерации оттенков исходного цвета и определения контрастности текста относительно этого цвета, используемого в качестве фонового.

Приложение позволяет пользователю добавлять один или несколько цветов, получать для каждого цвета набор оттенков по яркости, рассчитывать контрастность текста на выбранном фоне, а также переводить цвет между различными цветовыми форматами.

Функциональные возможности

Приложение должно предоставлять следующие возможности:

- добавление одного или нескольких исходных цветов;
- ввод цвета в различных форматах: HEX, sRGB, CIELAB, LCH, OKLCH, CMYK;
- автоматическое определение и проверка корректности введённого цвета;
- перевод цвета из одного формата в другой;
- выбор формата отображения цвета через переключатель или выпадающий список;
- разбиение исходного цвета на набор оттенков по яркости;
- отображение палитры оттенков для каждого добавленного цвета;
- расчёт контраста между цветом текста и исходным цветом, используемым как фон;
- автоматический подбор подходящего цвета текста: белого или чёрного;
- отображение результата проверки контрастности по требованиям WCAG;
- удаление добавленного цвета;
- копирование значения цвета в выбранном формате.

Расчёт контраста выполняется по формуле WCAG, представленной в спецификации Web Content Accessibility Guidelines в определении contrast ratio: <https://www.w3.org/TR/WCAG21/#dfn-contrast-ratio>

$$K = \frac{L_1 + 0.05}{L_2 + 0.05},$$

где L_1 — относительная яркость более светлого цвета, L_2 — относительная яркость более тёмного цвета. Значение контраста находится в диапазоне от 1 : 1 до 21 : 1.

Для обычного текста минимальное рекомендуемое значение контраста составляет 4.5 : 1, для крупного текста — 3 : 1.

Вводимые пользователем данные

Пользователь может вводить следующие данные:

- исходный цвет, используемый для построения палитры и проверки контрастности;
- формат отображения результата;
- параметры генерации оттенков;
- цвет текста для ручной проверки контраста;
- название палитры.

Цвет может задаваться в одном из поддерживаемых форматов: HEX, sRGB, CIELAB, LCH, OKLCH или CMYK. Формат введённого цвета определяется автоматически.

Требования к обработке цвета

Система должна выполнять следующие операции:

- распознавать формат введённого цвета;
- проверять корректность значений компонентов цвета;
- приводить цвет к внутреннему универсальному представлению;
- выполнять конвертацию между поддерживаемыми форматами;
- отображать результат конвертации без потери читаемости;
- корректно обрабатывать цвета с высокой и низкой яркостью;
- корректно работать с несколькими цветами одновременно.

Для расчёта контраста цвета приводятся к цветовому пространству sRGB, так как формула WCAG использует относительную яркость, вычисляемую на основе sRGB-компонентов.

Формула относительной яркости для sRGB приведена в определении relative luminance спецификации WCAG: <https://www.w3.org/WAI/WCAG21/Understanding/relative-luminance.html>

Относительная яркость вычисляется по формуле:

$$L = 0.2126R + 0.7152G + 0.0722B,$$

где R , G , B — линейные значения красного, зелёного и синего каналов.

Для перевода компонента sRGB в линейное значение используется формула:

$$C_{\text{linear}} = \begin{cases} \frac{C_{\text{srgb}}}{12.92}, & C_{\text{srgb}} \leq 0.04045, \\ \left(\frac{C_{\text{srgb}} + 0.055}{1.055} \right)^{2.4}, & C_{\text{srgb}} > 0.04045. \end{cases}$$

Особенности конвертации цветов

Для обеспечения стабильной конвертации приложение должно хранить исходное значение цвета, введённое пользователем, и использовать его как основу для последующих преобразований в другие форматы. При переключении формата отображения приложение должно использовать уже сохранённое значение в нужном формате, если оно есть в цепочке преобразований; если такого значения нет, оно должно быть получено конвертацией из актуального исходного цвета.

Изменение значения после конвертации фиксируется как задание нового цвета. Аналогично, если при преобразовании цвет был ограничен допустимым диапазоном целевого пространства, дальнейшие операции выполняются уже с этим ограниченным значением, поэтому обратное преобразование может не восстановить исходный цвет.

Например, если исходный цвет был задан в OKLCH и затем отображён в sRGB, приложение всё равно хранит исходное OKLCH-значение и может использовать его при дальнейших переключениях формата. Однако если пользователь изменит уже отображённое sRGB-значение, оно будет считаться новым исходным цветом. В таком случае обратное преобразование в OKLCH может не совпасть с первоначальным значением.

Разбиение цвета на оттенки

Для каждого добавленного цвета приложение должно строить палитру оттенков по яркости. Исходный цвет включается в эту палитру как

один из её оттенков, а остальные оттенки формируются относительно него.

Разбиение цвета выполняется в цветовом пространстве OKLCH или LCH, так как эти пространства позволяют удобно изменять воспринимаемую яркость цвета. При генерации оттенков изменяется компонент яркости, а тон и насыщенность по возможности сохраняются. Если исходный цвет находится близко к границе диапазона яркости, часть соседних оттенков может быть визуально близкой к нему.

Пример шкалы оттенков представлен на рисунке 1.



Рис. 1: Пример шкалы оттенков

Светлые значения шкалы соответствуют более ярким оттенкам, тёмные значения — более тёмным оттенкам. Исходный цвет может располагаться на разных позициях шкалы в зависимости от собственной яркости.

Для каждого оттенка необходимо отображать:

- визуальный образец цвета;
- значение цвета в выбранном формате;
- рекомендуемый цвет текста;
- коэффициент контраста с белым текстом;
- коэффициент контраста с чёрным текстом;
- статус соответствия требованиям WCAG.

Требования к контрасту

Для каждого цвета или оттенка система должна рассчитывать контраст:

- с белым текстом;
- с чёрным текстом;
- с пользовательским цветом текста, если он задан вручную.

Результат проверки контраста должен отображаться в понятном виде и содержать коэффициент контраста, рекомендуемый цвет текста и статус соответствия требованиям WCAG.

Критерии оценки контрастности:

- $K \geq 7$ — соответствует уровню AAA для обычного текста;
- $K \geq 4.5$ — соответствует уровню AA для обычного текста;
- $K \geq 3$ — подходит для крупного текста;
- $K < 3$ — недостаточный контраст.

Интерфейс приложения

Интерфейс приложения должен быть выполнен в виде рабочей области с цветовыми палитрами. При большом количестве добавленных цветов или сгенерированных оттенков интерфейс должен использовать прокрутку, чтобы палитры не выходили за границы рабочей области и оставались удобными для просмотра.

Основные области интерфейса:

1. Панель ввода цвета.

Содержит поле ввода цвета, кнопку добавления цвета и настройки генерации оттенков. Формат введённого цвета определяется автоматически.

2. Область цветовых палитр.

Содержит все добавленные пользователем цвета. Для каждого цвета отображается исходное значение, набор оттенков и результаты проверки контрастности. Пользователь должен иметь возможность расширять или сужать цветовую палитру, добавляя дополнительные оттенки вручную или удаляя лишние. Добавленные оттенки должны отображаться вместе с автоматически сгенерированными оттенками и участвовать в расчёте контрастности и конвертации форматов.

3. Блок контраста.

Отображает результаты проверки контрастности текста относительно исходного цвета, используемого в качестве фонового.

4. Блок конвертации.

Позволяет выбрать формат отображения цвета с помощью переключателя или выпадающего списка с готовыми вариантами: HEX, sRGB, CIELAB, LCH, OKLCH, CMYK. После выбора все цвета и оттенки отображаются в выбранном формате.

Поле ввода цвета располагается в верхней части интерфейса по центру. Основную часть экрана занимает область цветовых палитр. Блок

конвертации располагается слева от соответствующей цветовой палитры и применяется к цветам и оттенкам этой палитры. Блок контраста располагается сверху от цвета или оттенка, для которого выполняется проверка контрастности.

Технологический стек

Frontend-часть приложения реализуется с использованием Vue 3.

Backend-часть приложения реализуется на языке Python с использованием FastAPI.

Для работы с цветами используется библиотека ColorAide. Она применяется для парсинга введенных пользователем цветов, конвертации между цветовыми пространствами и генерации оттенков через изменение компонента яркости.

Расчёт контрастности выполняется отдельно по формуле WCAG на основе относительной яркости sRGB.

Требования к данным

Ограничения на вводимые данные:

- длина строки цвета — до 64 символов;
- длина названия палитры — до 80 символов;
- поддерживаемые форматы: HEX, sRGB, CIELAB, LCH, OKLCH, CMYK;
- некорректные значения должны сопровождаться понятным сообщением об ошибке.

Требования к хранению состояния

Приложение должно хранить:

- список добавленных цветов;
- выбранный формат отображения;
- настройки генерации оттенков;
- пользовательский цвет текста для проверки контраста.

Состояние приложения хранится во frontend-части. Дополнительно допускается сохранение данных в `localStorage`, чтобы после перезагрузки страницы добавленные цвета не удалялись.

История разработки

Ниже приведена история запросов на разработку, ответов и консольных выводов, начиная с запроса на добавление цветового пространства CIELAB и создание реализации приложения.

Запрос 1. Добавление CIELAB и первичная реализация

Запрос пользователя:

добавь в техническое задание цветовое пространство CIELAB

и реализуй техническое задание, создай папку src в них создай папки

- ↪ frontend и backend, инициализируй vue-проект через дефолтную команду
- ↪ и перепиши шаблон под нашу нужды (перед этим переключи версию node js на lts)

в backend запусти виртуальное окружение анаконды и подними сервер, сделай

- ↪ REST api, который работает в цветах, во frontend в .env прокинь наш
- ↪ api

Ответ:

Готово.

Добавил CIELAB в ТЗ в списки поддерживаемых форматов.

Создал реализацию:

- src/frontend -- Vue-проект, создан через npm create vue@latest, шаблон
 - ↪ переписан под палитры, конвертацию, оттенки и контраст.
- src/frontend/.env -- VITE_API_URL=http://127.0.0.1:8000/api.
- src/backend -- FastAPI REST API на ColorAide, поддерживает HEX, sRGB,
 - ↪ CIELAB, LCH, OKLCH, CMYK.
- src/backend/.conda -- conda-окружение создано и используется для
 - ↪ backend.
- .nvmrc -- выставлен lts/*; текущий Node переключён на LTS v24.16.0.

Проверил:

- npm run build во frontend проходит.
- Backend компилируется.
- GET /api/health отвечает {"status":"ok"}.
- POST /api/colors/analyze возвращает CIELAB, оттенки и контраст.

Серверы сейчас подняты:

- Frontend: http://127.0.0.1:5173/
- Backend API: http://127.0.0.1:8000/api

В git status ещё видны уже существующие изменения .DS_Store и

- ↪ report/main.pdf; я их не трогал.

Консольный вывод:

```

$ node -v
v12.22.12

$ npm -v
6.14.16

$ conda --version
conda 26.1.1

$ source ~/.nvm/nvm.sh && nvm install --lts && nvm use --lts && npm
  ↳ create vue@latest frontend -- --default
Installing latest LTS version.
Downloading and installing node v24.16.0...
Now using node v24.16.0 (npm v11.13.0)
Now using node v24.16.0 (npm v11.13.0)
npm warn exec The following package was not found and will be installed:
  ↳ create-vue@3.22.3

> npx
> "create-vue" frontend --default

- Vue.js - The Progressive JavaScript Framework

Scaffolding project in
  ↳ /Users/ilyasemenov/poly/4sem/digital-analytics/frontend...
|
- Done. Now run:

    cd frontend
    npm install
    npm run dev

$ npm install
added 116 packages, and audited 117 packages in 18s
30 packages are looking for funding
found 0 vulnerabilities

$ conda create -p
  ↳ /Users/ilyasemenov/poly/4sem/digital-analytics/src/backend/.conda
  ↳ python=3.11 -y
2 channel Terms of Service accepted
Retrieving notices: done
Solving environment: done
Executing transaction: done

$ conda run -p
  ↳ /Users/ilyasemenov/poly/4sem/digital-analytics/src/backend/.conda
  ↳ python -m pip install fastapi uvicorn coloraide
Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0
  ↳ anyio-4.13.0 click-8.4.1 coloraide-8.8.1 fastapi-0.136.3 h11-0.16.0
  ↳ idna-3.17 pydantic-2.13.4 pydantic-core-2.46.4 starlette-1.2.0
  ↳ typing-extensions-4.15.0 typing-inspection-0.4.2 uvicorn-0.48.0

```

```
$ conda run -p
→ /Users/ilyasemenov/poly/4sem/digital-analytics/src/backend/.conda
→ python -c "from coloraide.everything import ColorAll as Color;
→ c=Color('#7b61ff'); print(c.convert('cmyk').to_string())"
color(--cmyk 0.51765 0.61961 0 0)
```

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 393ms
```

```
$ curl -s http://127.0.0.1:8000/api/health
{"status":"ok"}
```

```
$ curl -s -X POST http://127.0.0.1:8000/api/colors/analyze -H
→ 'Content-Type: application/json' -d
→ '{"value":"#7b61ff","output_format":"CIELAB","shade_count":5}'
{"input":"#7b61ff","detected_space":"srgb","output_format":"CIELAB","hex":"#7b61ff","v
→ 41.67 -76.81)","formats":{"HEX":"#7b61ff","sRGB":"rgb(123 97
→ 255)","CIELAB":"lab(50.73 41.67 -76.81)","LCH":"lch(50.73 87.38
→ 298.5)","OKLCH":"oklch(0.6057 0.224 285.1)","CMYK":"color(--cmyk
→ 0.5176 0.6196 0
→ 0)"},"contrast":{"black":5.0,"white":4.2,"recommended":"black","recommended_hex":"
→ 1.844 -4.877)","formats":{"HEX":"#02000d","sRGB":"rgb(1.727 0
→ 13)","CIELAB":"lab(0.3257 1.844 -4.877)","LCH":"lch(0.3257 5.214
→ 290.7)","OKLCH":"oklch(0.08 0.0467 285.1)","CMYK":"color(--cmyk
→ 0.8671 1 0
→ 0.949)"},"contrast":{"black":1.01,"white":20.83,"recommended":"white","recommended
→ 42.33 -59.4)","formats":{"HEX":"#2d007c","sRGB":"rgb(45.04 0
→ 124.1)","CIELAB":"lab(14.45 42.33 -59.4)","LCH":"lch(14.45 72.94
→ 305.5)","OKLCH":"oklch(0.295 0.1723 285.1)","CMYK":"color(--cmyk
→ 0.637 1 0
→ 0.5134)"},"contrast":{"black":1.4,"white":14.97,"recommended":"white","recommended
→ 45.9 -76.76)","formats":{"HEX":"#6240dd","sRGB":"rgb(97.8 64.14
→ 221.3)","CIELAB":"lab(39.23 45.9 -76.76)","LCH":"lch(39.23 89.44
→ 300.9)","OKLCH":"oklch(0.51 0.224 285.1)","CMYK":"color(--cmyk 0.5581
→ 0.7102 0
→ 0.1321)"},"contrast":{"black":3.3,"white":6.36,"recommended":"white","recommended
→ 21.24 -51.73)","formats":{"HEX":"#9e97ff","sRGB":"rgb(157.6 150.7
→ 255)","CIELAB":"lab(66.3 21.24 -51.73)","LCH":"lch(66.3 55.92
→ 292.3)","OKLCH":"oklch(0.725 0.1489 285.1)","CMYK":"color(--cmyk
→ 0.3819 0.409 0
→ 0)"},"contrast":{"black":8.3,"white":2.53,"recommended":"black","recommended_hex"
→ 2.952 -10.77)","formats":{"HEX":"#e9e9ff","sRGB":"rgb(232.5 232.9
→ 255)","CIELAB":"lab(92.78 2.952 -10.77)","LCH":"lch(92.78 11.17
→ 285.3)","OKLCH":"oklch(0.94 0.0298 285.1)","CMYK":"color(--cmyk
→ 0.0882 0.0867 0
→ 0)"},"contrast":{"black":17.53,"white":1.2,"recommended":"black","recommended_hex"
```

```
$ npm run dev -- --host 127.0.0.1 --port 5173
VITE v8.0.14 ready in 160 ms
```

Local: http://127.0.0.1:5173/

Запрос 2. Конвертация внутри карточки, контраст, сохранение и заголовков

Запрос пользователя:

в карточке должна быть возможность перевести цвет в другой формат

поправь также в контраст у отображения контрастности

цвета должны сохраняться в localStorage и при перезагрузке страницы не
↪ удаляться

заголовок перенеси выше, добавь всем заголовкам свойство text-wrap:
↪ balance;

Ответ:

Готово.

В App.vue добавил:

- переключатель формата внутри каждой карточки;
- пересчёт конкретной карточки через REST API при смене формата;
- сохранение палитр в localStorage, чтобы после перезагрузки страницы они
↪ не удалялись.

В main.css поправил:

- заголовок теперь выше формы, а форма идёт отдельной строкой под ним;
- всем заголовкам добавлено text-wrap: balance;
- значения контрастности сделаны тёмными и читаемыми;
- добавлены стили для переключателя формата в карточке.

Проверка: npm run build проходит, локальный frontend отвечает HTTP/1.1
↪ 200 OK на http://127.0.0.1:5173/.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 121ms
```

```
$ curl -s -I http://127.0.0.1:5173/
HTTP/1.1 200 OK
```

Запрос 3. Редактирование цвета в карточке и стилизованный select

Запрос пользователя:

добавь возможность менять цвет в карточке, заголовок должен быть
→ одновременно input и при наведении или фокусе показывать, что он
→ input

и переключатель сделай лучше в виде стилизованного select

→ (https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/)

Ответ:

Сделал.

В App.vue:

- значение цвета в заголовке карточки теперь редактируемое input;
- по Enter или потере фокуса карточка пересчитывается через API;
- при ошибке ввода значение откатывается к последнему корректному;
- формат в верхней панели и в карточке заменён на стилизованный select.

В main.css:

- добавлены стили для select;
- заголовочный input выглядит как текст, но на hover/focus показывает
→ рамку и фон;
- убраны старые стили сегментированного переключателя.

Проверка: npm run build проходит.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 131ms
```

Запрос 4. Color-picker и выравнивание кнопок

Запрос пользователя:

в input добавь ещё color-picker (встроенный в браузер, конечно)

выровняй кнопки в карточке

Ответ:

Готово.

В App.vue добавил нативный input type="color":

- в верхнее поле ввода цвета;
- в редактируемый заголовок каждой карточки.

Для карточки выбор через color-picker сразу записывает HEX и

→ пересчитывает карточку через API.

В main.css поправил:

- стили нативного color-picker;
- выравнивание блока действий карточки: select и кнопка удаления теперь ↪ стоят по нижней линии и одной высоты;
- стили заголовочного input больше не ломают color-picker.

Проверка: npm run build проходит.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 146ms
```

Запрос 5. Снятие ограничения на оттенки и дополнительные picker-элементы

Запрос пользователя:

убери агрипичение на оттенки

picker тоже выровняй, добавь ему иконку

здесь нужны два picker-а
им тоже нужна иконка

Ответ:

Готово.

Изменил:

- снял верхнее ограничение на оттенки во frontend и backend; 20 теперь ↪ проходит;
- backend теперь корректно обрабатывает даже 1 оттенок;
- добавил второй color-picker для цвета текста;
- все pickers теперь выглядят как цветные кнопки с иконкой палитры;
- picker в карточке выровнен с полем формата и кнопкой удаления.

Проверил:

- npm run build проходит;
- backend компилируется;
- запрос с shade_count: 20 возвращает оттенок с "index": 20.

Backend перезапущен, чтобы новое ограничение применилось на
↪ http://127.0.0.1:8000/api.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 145ms

$ conda run -p
↳ /Users/ilyasemenov/poly/4sem/digital-analytics/src/backend/.conda
↳ python -m compileall app.py
Compiling 'app.py'...

$ curl -s -X POST http://127.0.0.1:8000/api/colors/analyze -H
↳ 'Content-Type: application/json' -d
↳ '{"value": "#827aa8", "output_format": "HEX", "shade_count": 20}' |
↳ python3 -m json.tool | rg '"index"' | tail -n 1
    "index": 20,
```

Запрос 6. Пользовательский цвет текста

Запрос пользователя:

если мы задали красный цвет теста, то во всех карточках тоже должен быть
↳ красный
и контраст должен считаться относительно красного и нет смысла писать в
↳ таком случае отдельную строку с контрастом

Ответ:

Готово.

Изменил логику так, что заданный цвет текста теперь применяется ко всем
↳ карточкам:

- при выборе красного через picker существующие карточки сразу
↳ пересчитываются;
- оттенки в карточках отображаются этим же красным цветом;
- контраст оттенков считается относительно красного, а не относительно
↳ рекомендованного белого/чёрного;
- если пользовательский цвет текста задан, блок контраста больше не
↳ показывает белый/чёрный/рекомендацию, а показывает только цвет текста
↳ и его контраст.

Backend тоже обновлён: теперь в ответе есть `text_color`, `contrast.text_hex`
↳ и `contrast.custom` для основного цвета и каждого оттенка.

Проверил:

- `npm run build` проходит;
- backend компилируется;
- API с `text_color: "#ff0000"` возвращает `text_hex: "#ff0000"` и `custom`
↳ `contrast` для оттенков.

Консольный вывод:

```

$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 137ms

$ conda run -p
  ↳ /Users/ilyasemenov/poly/4sem/digital-analytics/src/backend/.conda
  ↳ python -m compileall app.py
Compiling 'app.py'...

$ curl -s -X POST http://127.0.0.1:8000/api/colors/analyze -H
  ↳ 'Content-Type: application/json' -d
  ↳ '{"value":"#6c5eb0","output_format":"HEX","shade_count":3,"text_color":"#ff0000"}'
  ↳ | python3 -m json.tool | rg '"text_color"|"custom"|"text_hex"'
      "custom": 1.36,
      "text_hex": "#ff0000",
      "text_color": "#ff0000",
          "custom": 5.21,
          "text_hex": "#ff0000",
          "custom": 1.5,
          "text_hex": "#ff0000",
          "custom": 3.34,
          "text_hex": "#ff0000",

```

Запрос 7. Отображение рекомендаций без пользовательского цвета

Запрос пользователя:

но, если цвет текста не задан, как раз есть смысл в делении на контраст

- ↳ белого и чёрного и рекомендации по цвету текста

Ответ:

Да, именно так и должно работать.

Я поправил условие в App.vue: теперь если цвет текста не задан, карточка

- ↳ показывает контраст с белым, контраст с чёрным и рекомендацию. Если
- ↳ цвет текста задан, показывается только контраст с этим
- ↳ пользовательским цветом.

Проверка: npm run build проходит.

Консольный вывод:

```

$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 133ms

```

Запрос 8. Customizable select

Запрос пользователя:

и попробуй сделать стилизованный select

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Customizable_select

Ответ:

Сделал стилизованный select по подходу MDN customizable select.

Что изменил:

- В App.vue добавил внутрь <select> структуру:

```
<button>
  <selectedcontent></selectedcontent>
</button>
```
- В main.css добавил стили через:
 - appearance: base-select;
 - ::picker(select);
 - ::picker-icon;
 - option::checkmark;
 - option:checked.

Сделал это через @supports (appearance: base-select), потому что MDN
↪ прямо отмечает ограниченную поддержку этой технологии. В браузерах
↪ без поддержки останется текущий стилизованный select с обычным
↪ выпадающим списком.

Проверка: npm run build проходит.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 143ms
```

Запрос 9. Приведение select к дизайну интерфейса

Запрос пользователя:

хорошо, только select под дизайн нашего интерфейса и поправь расположение
↪ стрелочки

Ответ:

Готово.

В main.css поправил стилизованный select:

- выпадающее меню теперь в стиле интерфейса: белый фон, зелёный выбранный
→ пункт, мягкая тень, радиус 8px;
- убрал тёмно-розовую палитру;
- поправил стрелку в закрытом состоянии: она теперь ниже, ровнее и не
→ прижимается к верхнему краю;
- для base-select тоже поправил ::picker-icon.

Проверка: `npm run build` проходит.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 136ms
```

Запрос 10. Количество оттенков внутри карточки

Запрос пользователя:

добавь возможность изменения числа оттенков в карточке цвета

Ответ:

Готово.

В карточку добавил поле Оттенки рядом с форматом карточки и кнопкой
→ удаления. Теперь:

- каждая карточка хранит своё количество оттенков;
- изменение числа оттенков пересчитывает только эту карточку;
- значение сохраняется в `localStorage`;
- работает по `Enter` и по потере фокуса.

Изменения в `App.vue` и `main.css`.

Проверка: `npm run build` проходит.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 179ms
```

Запрос 11. Позиционирование стрелки select

Запрос пользователя:

задай иконке просто абсолютное позиционирование, а селекту по центру
и спозиционируй по центру селекта

Ответ:

Готово.

В main.css:

- стрелка select теперь позиционируется абсолютно;
- значение внутри select выровнено по центру по вертикали;
- для base-select стрелка тоже вынесена в absolute через ::picker-icon;
- fallback-стрелка тоже центрируется через translate.

Проверка: npm run build проходит.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 136ms
```

Запрос 12. Центрирование стрелки select

Запрос пользователя:

передвинь по центру

Ответ:

Сдвинул стрелку select ниже и привязал её к вертикальному центру самого
→ поля через top: 50%.

Изменено в main.css:

- fallback select-field::after;
- base-select ::picker-icon.

Проверка: npm run build проходит.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 123ms
```

Запрос 13. Позиционирование select

Запрос пользователя:

только селекту задай position: relative;

Ответ:

Добавил `position: relative` самому `select` в `main.css`.

Проверка: `npm run build` проходит.

Консольный вывод:

```
$ npm run build
vite v8.0.14 building client environment for production...
OK 1735 modules transformed.
OK built in 139ms
```